# Room as Code

## A room you can keep editing.

Describe a change. Keep the scene. Preview immediately. Render when ready.

Room as Code is an open-source project for creating and maintaining residential 3D scenes through conversation. Its current implementation, **Room Studio**, runs as a shared skill in Codex and Claude Code, with local Python tools, a browser preview and Blender exports.

The central idea is simple: every edit should become a deliberate change to a saved scene, with unrelated objects and existing material choices preserved.

**PROJECT GUIDE** · PROTOTYPE 0.1 · 7 NOVEMBER 2026



The Manhattan demonstration apartment, rendered in Blender. Concealed lighting and the night environment are saved scene settings.

01 / PURPOSE

ROOM AS CODE

# Make the second change easier.

A useful 3D workflow continues after the first convincing image. A developer may need to move a sofa, raise an artwork, change the lighting or adapt a room for a new scene. The existing layout and approved details should remain available throughout that work.

Room as Code treats the home as persistent, inspectable data. A host agent interprets the request, local scripts validate a structured edit, and the preview reflects the saved result. Rendering is a separate, explicit step.

## Designed for developers building spaces

The initial audience is independent game developers, interactive-story creators and technical artists who need editable residential scenes and repeatable variations. The project is also a test bed for agent-driven work on structured 3D assets.

### 01 / Persistent state

Rooms, furniture and decoration have stable identities. Save the scene, reopen it, and continue from the same state.

### 02 / Local, reviewable edits

Change the intended objects, inspect the field differences, preserve explicit protections, and undo a transaction when needed.

### 03 / A short feedback loop

Use the live browser preview while editing. Generate finished images after a batch of changes has settled.

## The proposed distinction

The product direction is **conversational scene maintenance**: a readable scene state, explicit edits and reproducible outputs. This is a development thesis to validate with users; the project does not claim proven market differentiation or automatic reconstruction from photographs.

**Two names, one implementation.** Room as Code is the public project name. The current skill, commands, plugin and scene schema retain the compatible name `room-studio`.

02 / INPUTS

ROOM AS CODE

# Bring what you have.

The recommended input is a floor plan, reliable dimensions, a small set of consistent visual references and a written brief. Each source contributes a different kind of information.

| INPUT | WHAT IT ESTABLISHES | USEFUL DETAIL |
|---|---|---|
| Floor plan | Boundaries, room relationships, doors, windows and circulation | A whole-apartment plan for a whole-home scene |
| Measurements | Scale and height | Dimensioned walls and ceiling height; at least one reliable length when information is limited |
| Style references | Materials, palette, furniture language and lighting intent | Identify which features to borrow from each image |
| Written brief | Use, priorities and constraints | Occupants, room functions, objects to retain and changes to avoid |
| Site photographs | Visible conditions in an existing space | Multiple useful views when recreating a real interior |

## Incomplete material is a valid starting point

**Text and style images only:** create a design study with clearly stated assumptions. **Photographs without a plan:** model visible objects and supported spatial relationships, keeping unseen areas uncertain. **A plan without dimensions:** establish topology first, then calibrate its tentative scale using reliable measurements.

Record the difference between a measurement, a photo observation, a user estimate and a design assumption. A user's approval does not turn an estimate into a measurement. Floor area alone does not establish every wall length.

> “Create a three-bedroom apartment from this plan. Use the reference images for warm wood and light fabrics. Keep the dining table in place, use concealed lighting, and mark any dimensions you have to assume.”

**Current input boundary.** The image-capable host agent interprets references and authors or adapts scene data. The bundled scripts do not automatically recover an accurate floor plan, camera pose or hidden geometry from uploaded images. `init` creates the supplied demonstration home.

03 / WORKFLOW

ROOM AS CODE

# From intent to a saved scene.

The agent handles language. Deterministic local tools handle supported scene changes. Both hosts use the same scene protocol and runtime scripts.

01 **Inspect**

Read the current scene, revision, object labels, stable IDs, parameters and material overrides.

02 **Resolve the request**

The host identifies the objects, units, direction, scope and protected items. Ask when a missing detail would change the result.

03 **Validate and save**

A structured patch carries the expected revision. The transaction validates every operation before writing the new state.

04 **Compile and preview**

The saved state produces deterministic geometry. The local browser reads the same scene and refreshes its 3D view.

05 **Export or render**

Synchronize the managed Blender model when requested. Generate full images at an agreed final checkpoint.

## One scene, several representations

The scene JSON is the persistent working state. The browser is a lightweight view of that state. Blender is an editable native output with procedural materials, cameras and lights. GLB is a portable export with a simpler material representation.

## Explicit failure is part of the contract

Unsupported operations, invalid values and stale revisions are rejected. A failed tool call is not a successful edit. When detected external changes conflict with a managed Blender master, synchronization stops and preserves the existing file.

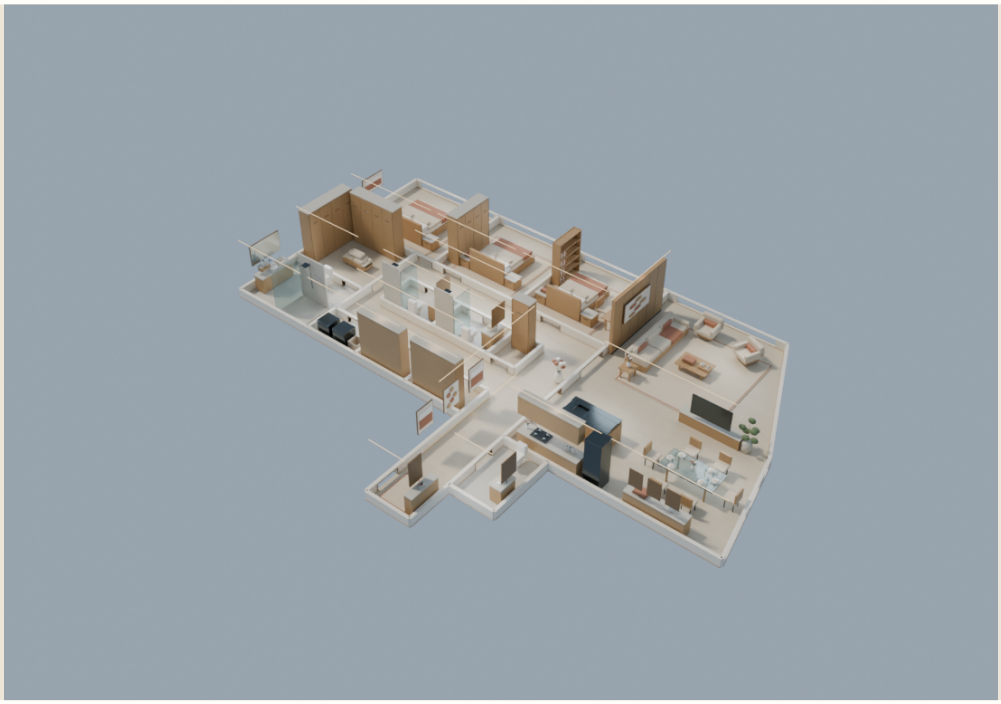
**The browser is a preview, not a second agent.** Natural-language understanding runs in Codex or Claude Code. The viewer has no independent chat model; its template controls use the same local scene state.

04 / DEMONSTRATION

ROOM AS CODE

# A complete apartment to work on.

The principal demonstration is a Manhattan-inspired home with an entrance foyer, long gallery, corner living room, connected kitchen and dining area, three bedrooms, three full bathrooms, a powder room and service spaces.



A cutaway view of the demonstration. Low walls reveal circulation; the saved Blender master retains full-height architecture.

| 267 m² | 19 | 251 | 2,189 |
|---|---|---|---|
| Design envelope | Named spaces | Editable entities | Native meshes |

## Structure first, finishing second

The layout and openings were frozen before the furnishing pass. The finishing adds wood wall panels, upholstered furniture, rugs, bedding, framed art, books, table settings, ceramics and bathroom storage. The sample uses recessed or concealed lighting.

The spatial organization was studied from the published developer plan for **35 Hudson Yards, Residence 02, floors 75–82**. The model is a parametric design adaptation. Its approximately 267 m² envelope is not a surveyed net area or a construction drawing; the reference plan itself distinguishes its stated area from usable area. No balcony was added to this case.

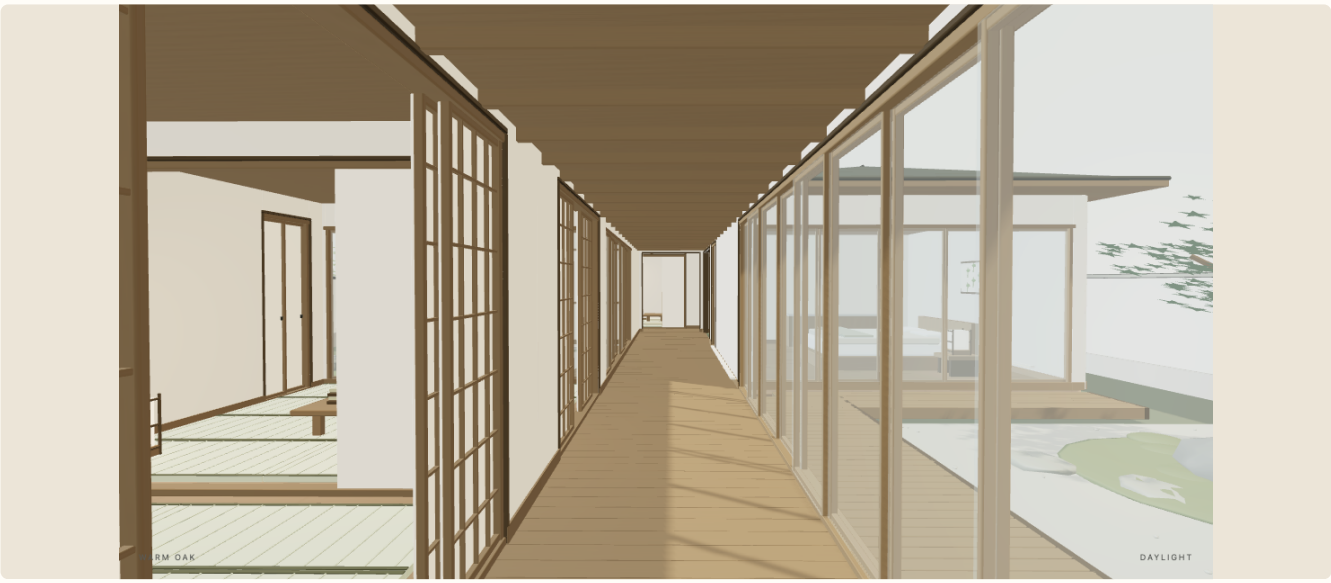
Source: Developer floor-plan PDF preserved by LX Collection. The third-party drawing is not included in the distributable sample.

05 / JAPANESE RESIDENCE

ROOM AS CODE

# Three gardens, one quiet home.

**NIWA TO MA** is the second residential sample: a Japanese single-storey house with three open-to-sky gardens and an engawa corridor. Tatami rooms, shoji screens, low furniture and warm timber connect the indoor spaces to planting and gravel.




The engawa and courtyard in the interactive WebGL preview. This is a browser capture of the generated scene, not a native Blender render.

| 221.51 m² | 3 | 241 | 3,033 |
|---|---|---|---|
| Indoor design area | Open gardens | Editable entities | Mesh parts |

## Switch samples without mixing their edits

Run `python3 preview_samples.py --port 8765`, then use the sample selector. Each launch makes fresh working copies in a new `runs/` directory. Both examples use the same preview, patch validation and undo/redo system. Japanese screens change actual geometry; garden controls open the three saved views.

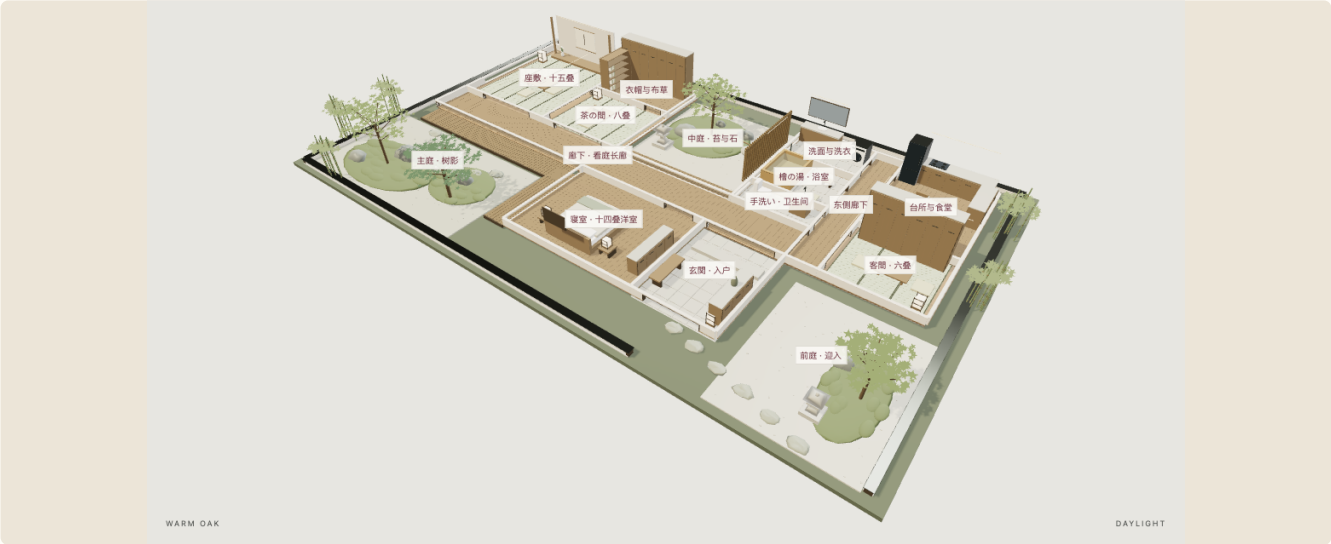
The layout study used a published Neyagawa property plan. Its listing gives 221.51 m²; room dimensions, roof forms, fittings and planting are design estimates. The model preserves the studied adjacencies rather than claiming measured reconstruction. The original flyer and reference images are excluded; see third-party notices for source attribution.

06 / COURTYARDS & LAYOUT

ROOM AS CODE

# Read the house through its gardens.

The front garden, central courtyard and main garden remain distinct outdoor rooms. The engawa provides the long circulation edge linking the quiet interior to them. Roof visibility and cutaway views help inspect these relationships.




The complete Japanese sample in the shared browser preview. The overview removes roofs for inspection while retaining the three garden spaces.




The central courtyard: gravel, moss, stepping stones and a maple. Planting and built details are generated geometry with stable entity IDs.

07 / EDITING

ROOM AS CODE

# Say what should change.

Use natural language to select an object and describe a change. The host translates that request into supported operations and verifies the saved result.

**Request**

“Move the living-room sofa 30 cm along world X and rotate it another 15 degrees. Raise the wall artwork by 15 cm. Keep the existing fabric and leave the dining table untouched.”

**Expected behavior**

Resolve the sofa and wall artwork by their saved identities. Copy untouched coordinates exactly. Apply the transform changes together, retain the material override, and verify the protected dining table. Report the changed entities and field differences.

| CAPABILITY | CURRENT BEHAVIOR |
|---|---|
| Targeted changes | Position, rotation, supported dimensions, visibility and material-color overrides |
| Explicit protection | A patch can name entities that must remain unchanged |
| Revision checks | An edit based on an old revision is rejected |
| Atomic writes | Invalid operations reject the transaction rather than saving a partial edit |
| Undo and redo | Restore scene content, with revision numbers continuing to advance |
| Incremental native sync | Reuse unchanged managed primitives and stop on detected external conflicts |

## Check the dependencies you actually intend to move

A countertop's supported parametric subparts regenerate from its dimensions. Independently positioned props are separate entities: moving a sofa does not automatically move every nearby object or refit its throw. Include related items in the request and inspect the result.

“Left” also requires a known camera orientation or an explicit world direction.

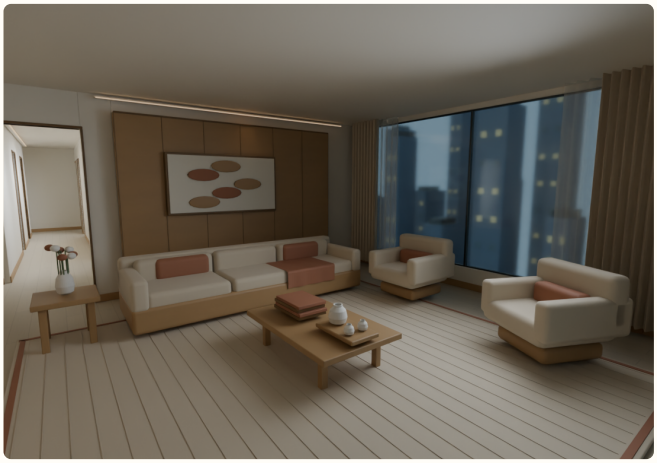
Selection uses stable identity, labels and detail metadata. A generic geometry family alone cannot reliably distinguish a wall picture from a tabletop accessory.

08 / ATMOSPHERE

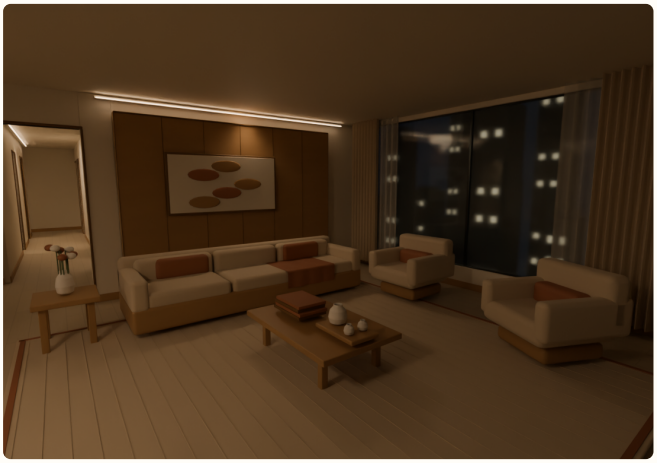
ROOM AS CODE

# Keep the layout. Change the atmosphere.

Three material themes share the same scene: **Warm**, **Industrial Metal** and **Heritage**. A theme can apply globally or to a room. Object color overrides remain available, and room overrides survive a global theme change.



Day / sky and sun with the saved interior setup.



Night / concealed lighting with the night environment.

| LAYER | WHAT CAN CHANGE | WHAT REMAINS INDEPENDENT |
|---|---|---|
| Material theme | Effective palette across supported surfaces | Layout and saved object overrides |
| Day or night | Environment illumination and actual light sources | The selected material style |
| Window environment | Procedural city, garden or sea setting | The day/night selection |
| Curtains | Pleated geometry and opening amount | Other furniture and materials |
| Concealed lighting | On/off, brightness and color temperature | Curtain and outdoor settings |

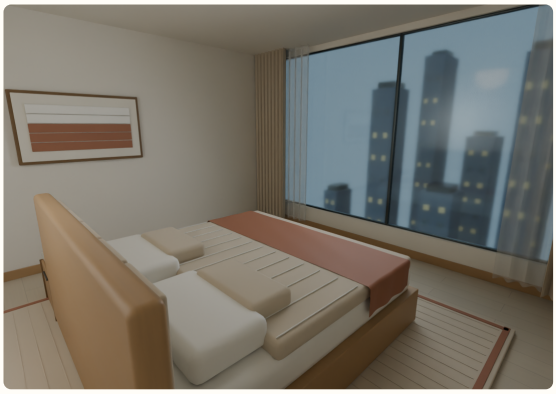
## Two visual outputs, with different fidelity

The browser approximates transmission, reflections and indirect light for interactive feedback. Blender performs the separate full render. Curtains are parametric fabric forms, and the city is a designed procedural environment; neither is a physical simulation or a Manhattan location capture.

09 / SCENE & EXPORTS

ROOM AS CODE

# Keep the scene. Choose the output.



The principal bedroom after the finishing pass. The coverlet is a shaped mesh with a supported contact area and hanging edges, not a simulated cloth system.

| ARTIFACT | ROLE IN THE WORKFLOW |
|---|---|
| Scene JSON | Persistent entities, rooms, parameters, overrides, revision and edit history |
| Browser preview | Interactive inspection and saved style, curtain and lighting controls |
| Blender `.blend` | Editable native geometry, cameras, lights and full procedural surface materials |
| `.glb` | Portable geometry with explicit constant PBR material fallback |
| PNG + `.render.json` | A rendered image and the scene/image information needed to check freshness |

## Full materials stay in Blender

The native adapter includes wood grain, floor boards, fabric weave and stone micro-surfaces while preserving effective color choices. GLB currently receives colors and supported scalar material properties. Procedural texture detail is not baked into GLB.

## Saved images have a version

`render-status` reports **CURRENT**, **STALE** or **MISSING**. A later scene change, a replaced image or a changed native material version can invalidate an old render. Editing during rendering can also make the new image stale. Returning to identical scene content through undo can restore a matching image's current status.

10 / RUN LOCALLY ROOM AS CODE

# Run locally. Continue in either host.

| REQUIREMENT | SCOPE |
|---|---|
| Python 3.10+ | Local scene tools; the runtime core uses the standard library |
| A WebGL2 browser | Interactive preview; built frontend assets are included |
| Blender | Native export and full rendering; validated with 4.2.23 LTS |
| Codex or Claude Code | The host agent that interprets conversation |

## 1 / Install the shared skill

From the extracted source directory, replace the destination with your project:

```
python3 install_skill.py --host codex --project /path/to/project
python3 install_skill.py --host claude --project /path/to/project
```

Choose the host you use. Codex installs under `.agents/skills/room-studio/`; Claude Code uses `.claude/skills/room-studio/`. The installer preserves existing installations. In Codex, invoke `$room-studio`; in Claude Code, select the skill from the host's available skills.

## 2 / Create and preview a working scene

```
ROOM=skills/room-studio/scripts/room.py
python3 "$ROOM" doctor
python3 "$ROOM" init my-house.scene.json
python3 "$ROOM" serve my-house.scene.json --port 8766
```

Open the loopback address reported by the server. Continue editing the same file in your host conversation. A new `init` creates the fixed demonstration home, not an automatic reconstruction of your references.

## 3 / Render after the edits are settled

```
python3 "$ROOM" render my-house.scene.json \
  --out output/house.blend --image output/living.png --room living
python3 "$ROOM" render-status my-house.scene.json output/living.png
```

For a native export without rendering, use `blender` instead of `render` and omit `--image`. If automatic Blender discovery fails, set `BLENDER` to the existing executable. npm is needed only when rebuilding the frontend.

11 / VALIDATION ROOM AS CODE

# Evidence, with its scope intact.

The project separates unit checks, host execution, browser inspection and native-model verification. Passing one is not presented as proof of all the others.

| VERIFICATION | ESTABLISHED RESULT |
|---|---|
| Python integration | 35 tests passed across transactions, persistence, history, interfaces, layout and render provenance |
| Real host execution | Codex and Claude Code each installed the skill, moved a sofa and fixed artwork, preserved unrelated state, and checked undo/redo across processes |
| Current apartment | Full 251-entity case exported to 2,189 native meshes; save/reopen, GLB reimport, two-object local changes and undo passed |
| Real browser | WebGL2 navigation, room views, saved controls, narrow layout and error checks passed |
| Native materials | Upgrade, reuse, user colors and shader-node conflict checks passed; GLB fallback was inspected separately |
| Final images | Living room by day and night, principal bedroom and corrected cutaway reviewed; source status CURRENT |
| Distribution | Source and skill archives passed CRC, per-file hashes and clean installation checks |

**Host-test scope:** the real Codex and Claude Code editing runs used a historical 138-entity fixture. The new Manhattan case received separate full-scene native and browser integration checks. Those are complementary tests, not the same end-to-end run.

## Rendering time is a separate budget

One local run at 1,100 × 770 pixels and 32 samples measured approximately **79 seconds** for the living room by day, **119 seconds** at night and **129 seconds** for the bedroom. The corrected cutaway took about **11 seconds**. These include startup, synchronization, export and image generation; they are not per-edit latency promises.

Host/model waiting time is separate from local script time. Everyday edits do not automatically pay the full rendering cost.

Evidence index: `docs/VALIDATION.md`; native and visual records: `docs/validation/final-native-results.json` and `docs/validation/final-visual-results.json`. Fresh publication checks: `docs/validation/publication-checks.json`. Archive verification is generated at `dist/release-validation.json`.

12 / SCOPE & PACKAGES

ROOM AS CODE

# A useful prototype, with clear edges.

The current release demonstrates a maintained scene and a reproducible editing loop. It does not yet provide automatic accurate photo reconstruction, arbitrary furniture generation, reliable recovery of hidden geometry or a surveyed building model.

New object types, complex geometry and arbitrary image textures require generator work. Externally edited Blender topology and materials are not automatically merged; detected conflicts stop synchronization. Parametric textiles need contact checks when supporting furniture changes.

## Two packages, two residential samples

| PACKAGE | CONTAINS | START HERE WHEN YOU WANT TO... |
|---|---|---|
| Source | Runtime, preview source and bundle, tests, documentation and generated examples | Develop, extend or inspect the project |
| Skill | Host instructions, runtime scripts, references and preview assets | Use conversation in Codex or Claude Code |

The public archives are `room-as-code-0.1.0-source.zip` and `room-studio-0.1.0-skill.zip`. The current main branch adds both residential samples, the sample-switching preview and Japanese browser captures. The original v0.1.0 downloads preserve their historical baseline. Build the current source archive to include both samples; use the documented export commands for native Blender and GLB files. Original code is MIT-licensed and dependency notices are included. Private photographs and third-party reference drawings are excluded.

## What to validate next

1. **Input handling:** make incomplete references and uncertain dimensions easier to review, without pretending they are measurements.
2. **Scene relationships:** improve attached-object and support relationships so furniture changes can update dependent details reliably.
3. **Real projects:** test repeated editing with developers' own rooms, recording lost overrides, corrective work and failed operations.

These are proposed next steps, not features claimed for this release. There are no user-study results establishing product-market fit.

**Start with the case. Keep working with the skill. Extend with the source.** The goal is a room that remains understandable and editable after the first image.

Room as Code / Built with Room Studio 0.1 / Project guide, 7 November 2026.